TARI OOTLE · WOOCOMMERCE INTEGRATION

Ootle Payment Gateway for WordPress Complete Setup Guide

Everything a new user needs to go from a blank server to a working WooCommerce checkout that accepts TARI — wallet daemon install, API key minting, plugin configuration, webhook connection, and first test payment.

SETUP GUIDE | WOOCOMMERCE | TARI OOTLE L2 | OCTOBER 2026

4	11	6	60–90
COMPONENTS	SECTIONS	WEBHOOK EVENTS	MIN SETUP TIME

This guide assumes **no prior Tari or Ootle experience**. It starts from downloading the wallet daemon on a blank server and ends with a completed test payment in WooCommerce. Every link is clickable. Every command is copy-paste ready.

> **BEFORE YOU START**
>
> You need: a Linux/macOS/Windows server with a shell · PHP 7.4+ with curl enabled · WordPress 6.0+ with WooCommerce 6.0+ installed · outbound HTTPS access to the Ootle indexer and your wallet daemon · testnet tTARI (free from the faucet) or real TARI for production.

---

1 Overview — how the pieces fit together

The plugin is one of three moving parts. WooCommerce handles the storefront and orders. The **wallet daemon** holds your keys and signs transactions. The **indexer** reads on-chain state and streams events. Nothing works until all three are running and reachable.

FIGURE 1 — SYSTEM ARCHITECTURE: WOOCOMMERCE → WALLET DAEMON → OOTLE NETWORK



Checkout flow, end to end: customer clicks "Pay with Ootle" → plugin calls the wallet daemon's JSON-RPC to create a payment → customer pays on the Ootle network → an `OrderPlaced` event lands in the indexer → your webhook listener POSTs to `?wc-api=ootle_webhook` → the plugin verifies the signature and marks the order complete.

---

2 Install the Ootle wallet daemon

The wallet daemon is the only process that holds your private keys. It signs every transaction the gateway sends. Everything else is read-only. Run it somewhere your WordPress server can reach over the network, and nowhere else.

2.1 · DOWNLOAD THE WALLET DAEMON

The release ships a standalone `tari_ootle_walletd` binary per platform. **Note:** there is no `tari_ootle_wallet_cli` binary in the release — the CLI is not published as a prebuilt asset. You interact with the daemon directly over JSON-RPC (which is what the plugin does).

PLATFORM	ASSET FILENAME (V0.45.0)
Linux x86_64	`tari_ootle_walletd-0.45.0-9c62606-linux-x86_64`
Linux arm64	`tari_ootle_walletd-0.45.0-9c62606-linux-arm64`
macOS arm64 (Apple Silicon)	`tari_ootle_walletd-0.45.0-9c62606-macos-arm64`
Windows x64	`tari_ootle_walletd-0.45.0-9c62606-windows-x64.exe`

Get the current filenames and hashes from the releases page — the build hash (`9c62606`) changes with each release.

1 **Download the binary**

Grab the `tari_ootle_walletd` asset for your OS from the releases page.

2 **Verify the checksum**

Each asset has a matching `.sha256` file. Compare before running:

SHELL VERIFY THE DOWNLOAD Copy

```
# Linux example — paste the hash from the release's .sha256 file
echo "8cfdae03cc7dc7987752b2f1ae94046fc5c15258e1e26e0587977f7230712fae  tari_ootle_walletd-0.45
```

3 **Make it executable**

Run `chmod +x tari_ootle_walletd` in the download directory.

4 **Place it on your PATH**

Move it to `~/.local/bin/` or `/usr/local/bin/` (any directory already on PATH) so you can call it from anywhere.

2.2 · CHOOSE YOUR NETWORK

NETWORK	FLAG	USE CASE	FUNDS
esmeralda	`--network esmeralda`	Testing, development, staging (default)	FREE FAUCET
mainnet	`--network mainnet`	Live production payments	REAL TARI

START ON TESTNET

Mistakes on testnet cost nothing. Get the entire flow working on esmeralda first, then repeat the same steps on mainnet with a fresh, funded account and a production API key.

2.3 · START THE DAEMON

This command runs the daemon on the esmeralda testnet, bound to localhost so only this machine reaches it. The web UI is served on the same host at port 5100.

SHELL START THE WALLET DAEMON ON TESTNET Copy

```
tari_ootle_walletd --network esmeralda --listen-on 127.0.0.1:5100 --authentication none --enable-vite-dev-
```

OPTION	MEANING
`--network esmeralda`	Connect to the esmeralda testnet (the default). Use `mainnet` for production.
`--listen-on 127.0.0.1:5100`	Bind address. Use `127.0.0.1:5100` if the plugin runs on the same machine; `0.0.0.0:5100` to accept remote connections.
`--authentication none`	Disables WebAuthn passkey login so you can drive the daemon over JSON-RPC. **Test/disposable wallets only.**
`--enable-vite-dev-port 5173`	Sets the WebAuthn origin to `http://localhost:5173` and enables permissive CORS — fixes the "invalid domain" error in the web UI.
`Default ports`	5100 = JSON-RPC + web UI (the plugin connects here)

SECURITY WARNING YOU MUST READ

The command above runs without additional daemon-level authentication — appropriate only for a disposable testnet wallet. The daemon logs a loud warning saying so, and it is correct. **Never expose an unauthenticated daemon holding real value.** In production, keep the daemon bound to `127.0.0.1` or a private network, and front it with an HTTPS reverse proxy plus a firewall.

Open http://localhost:5100 in your browser to reach the wallet web UI — create accounts, check balances, claim testnet funds, and manage API keys from there.

---

3 Create an account and fund it

3.1 · CREATE A MERCHANT ACCOUNT

There is no wallet CLI binary in the release — you drive the daemon over its JSON-RPC endpoint. First bootstrap a bearer token (with `--authentication none`, this needs no passkey), then create the account.

SHELL BOOTSTRAP AN ADMIN TOKEN AND CREATE THE MERCHANT ACCOUNT Copy

```
# 1. Bootstrap an Admin bearer token (auth=none: no passkey, no WebAuthn)
TOKEN=$(curl -s -X POST http://127.0.0.1:5100/json_rpc -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"auth.request","params":{"permissions":["Admin"],"credentials":{"None":nul
  | python3 -c "import sys,json; print(json.load(sys.stdin)['result']['token'])")

# 2. Create the merchant account (returns the account + its Ootle address)
curl -s -X POST http://127.0.0.1:5100/json_rpc -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"jsonrpc":"2.0","method":"accounts.create","params":{"name":"merchant"},"id":2}' \
  | python3 -m json.tool
```

Note the `owner_public_key` and the `otl_esm_…` address from the response — you paste the public key into the plugin's **Merchant Public Key** field later.

> **WHY {"NONE":NULL}?**
>
> Under `--authentication none`, `auth.request` expects a tagged enum credential. `{"None":null}` is the variant that satisfies it — this is the documented bootstrap that replaces the WebAuthn passkey flow, and it is what makes the "invalid domain" error irrelevant for programmatic access.

3.2 · CLAIM TESTNET FUNDS (TESTNET ONLY)

Two ways, both free on esmeralda:

METHOD	HOW
Web UI	Click **"Claim Testnet Funds"** at http://localhost:5100
JSON-RPC	Call `accounts.create_free_test_coins` against the daemon (below)

The account argument is a tagged enum, not a bare name — pass the component address as `{"ComponentAddress":"…"}`. A claim also needs a `max_fee`.

SHELL CLAIM TESTNET FUNDS — Copy

```
ADDR="component_4293145b…"  # your account's component_address
curl -s -X POST http://127.0.0.1:5100/json_rpc -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d "{\"jsonrpc\":\"2.0\",\"method\":\"accounts.create_free_test_coins\",\"params\":{\"account\":{\"Compo
```

FAUCET CAN BE EMPTY — CHECK FIRST

The shared esmeralda faucet runs dry. Before claiming, check it: `accounts.get_faucet_balance` returns `balance` and `claim_amount`. If `balance < claim_amount` the claim fails with "the testnet faucet does not hold enough funds." That is a testnet top-up issue on Tari's side, not a client bug — ask in the Tari Discord #development channel and retry once refilled. The alternative is claiming tTARI by burning XTM on L1, which needs real funds.

3.3 · CONFIRM YOUR BALANCE

SHELL CONFIRM THE ACCOUNT HAS A NON-ZERO BALANCE Copy

```
curl -s -X POST http://127.0.0.1:5100/json_rpc -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d "{\"jsonrpc\":\"2.0\",\"method\":\"accounts.get_balances\",\"params\":{\"account\":{\"ComponentAddres
  | python3 -m json.tool
```

You must see a non-zero balance before moving on. An empty account cannot pay transaction fees, and every Ootle transaction pays fees in TARI.

---

4 Mint an API key

The plugin authenticates to the wallet daemon with a long-lived, permission-scoped API key. An Admin mints it once; the plugin then presents it as a `Bearer` token on every JSON-RPC call. There is no interactive login, no JWT exchange, and no session to refresh.

4.1 · HOW THE KEY WORKS

ASPECT	DETAIL
Format	32 random bytes, prefixed with `tw_`, base64-encoded
Shown	EXACTLY ONCE at creation — the daemon cannot display it again
Stored	Only a SHA-256 hash — a database leak does not expose the key
Sent as	`Authorization: Bearer tw_...` on every request
Revocation	Immediate — the next request after revoke fails, no cache

COPY IT THE MOMENT IT APPEARS

The raw key is displayed once, in the creation dialog, and then never again. Close the dialog and it is gone. Paste it directly into your password manager — not a text file, not a ticket, not chat.

4.2 · CHOOSE THE SMALLEST SCOPE SET

Grant only the permissions the plugin actually needs. An agent cannot self-promote: permissions come from the stored key row, not from anything the plugin sends.

PLUGIN ROLE	RECOMMENDED SCOPES
Monitoring only	`AccountInfo`, `TransactionGet`
Full payment gateway	`AccountInfo`, `TransactionSend_<your-account>`, `TransactionGet`

NEVER GRANT ADMIN BY DEFAULT

`Admin` lets the key mint, list, and revoke other keys. Grant it only with a deliberate reason, and only with the explicit confirmation flag. For a payment gateway, the transaction-send scope for your one merchant account is enough.

4.3 · CREATE THE KEY VIA WEB UI (OPTIONAL)

The web UI works too, but only if WebAuthn can reach it — open http://localhost:5100/api-keys and log in via passkey. If you hit an "invalid domain" error (WebAuthn rejects non-localhost origins), use the JSON-RPC method in 4.4 instead — it needs no browser at all.

1. **Navigate to API Keys**
   Go to the `/api-keys` route in the wallet web UI.

2. **Click Create API Key**
   Give it a friendly name, e.g. `wordpress-gateway`.

3. **Enter the scopes**
   Paste the comma-separated scope list from the table in 4.2.

4. **Set an expiry (recommended)**
   Prefer the shortest expiry that fits. A leaked short-lived key dies on its own without manual revocation.

5 **Copy the raw key immediately**

It appears once, with a copy button. Store it before dismissing the dialog.

4.4 · CREATE THE KEY OVER JSON-RPC (NO CLI, NO PASSKEY)

With `--authentication none`, mint the key directly against the daemon using the bootstrap token. The raw key prints to the response **once** — pipe it straight into your secret store.

SHELL MINT AN API KEY FOR THE WORDPRESS GATEWAY Copy

```
# Basic key with two scopes
curl -s -X POST http://127.0.0.1:5100/json_rpc -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"jsonrpc":"2.0","method":"auth.create_api_key","params":{"name":"wordpress-gateway","permissions":[
  | python3 -m json.tool

# With an expiry (unix seconds; None = never)
curl -s -X POST http://127.0.0.1:5100/json_rpc -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"jsonrpc":"2.0","method":"auth.create_api_key","params":{"name":"wordpress-gateway","permissions":[
  | python3 -m json.tool

# Granting Admin requires the explicit confirm_admin flag
curl -s -X POST http://127.0.0.1:5100/json_rpc -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"jsonrpc":"2.0","method":"auth.create_api_key","params":{"name":"full-access","permissions":["Admir
  | python3 -m json.tool
```

TASK	RPC METHOD
List keys	`auth.list_api_keys` — metadata only, never raw keys
Revoke	`auth.revoke_api_key` with `{"id":<n>}` — effective immediately

4.5 · WHAT THE DAEMON PROTECTS YOU FROM

- ✓ **Wrong, revoked, or unknown key** — identical error message and timing for all three; an attacker cannot enumerate valid hashes.
- ✓ **Out-of-scope request** — the stored scope set is enforced against every endpoint; the request fails with `InsufficientPermissions`.
- ✓ **Non-admin minting keys** — rejected; only `Admin` reaches the key-management endpoints.
- ✓ **Expired key** — silently filtered out; the next request fails exactly like a revoked key.

5 Install the plugin

1. **Log into WordPress admin**
   Any account with plugin-install privileges works.

2. **Go to Plugins → Add New → Upload Plugin**
   Choose the `ootle-payment-gateway.zip` file and click **Install Now**.

3. **Activate the plugin**
   Click **Activate Plugin** after installation completes.

4. **Confirm WooCommerce is active**
   The plugin requires it and silently does nothing without it. The gateway appears under WooCommerce → Settings → Payments automatically — no code edits needed.

> **WHAT ACTIVATION DOES**
>
> The plugin registers itself under gateway id `ootle` via the `woocommerce_payment_gateways` filter and flushes rewrite rules so the webhook endpoint resolves. Nothing else is written to your database until you save the settings.

---

6 Configure the plugin

Go to **WooCommerce → Settings → Payments**. You'll see the *Payment providers* list — find the **Ootle Payment Gateway** row (badged "Test mode") and click its **Manage** button to open the settings form. Fill in every field below, then **Save changes** at the bottom of the page.

FIGURE 2 — THE PAYMENTS PROVIDERS LIST; CLICK MANAGE ON THE OOTLE ROW

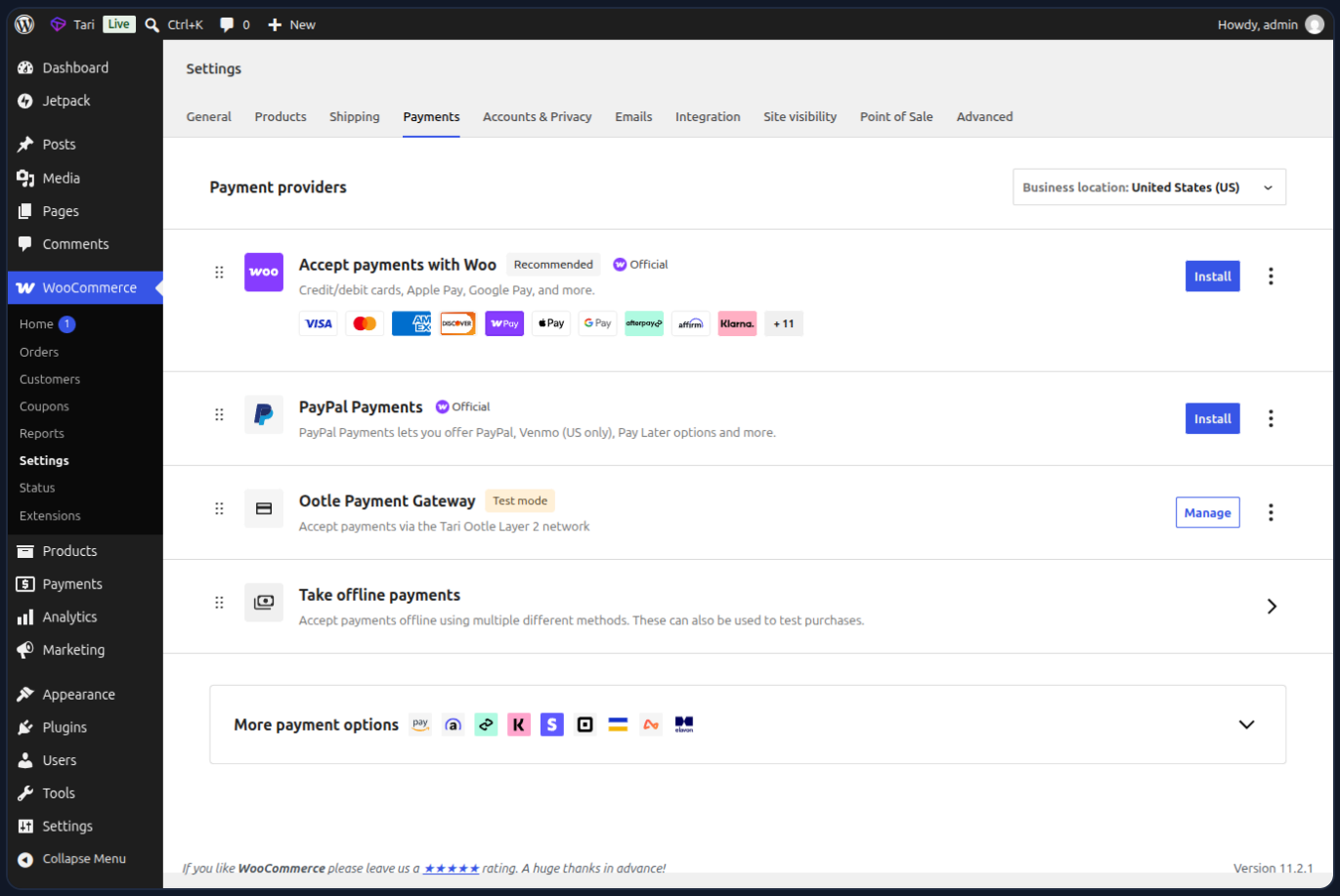
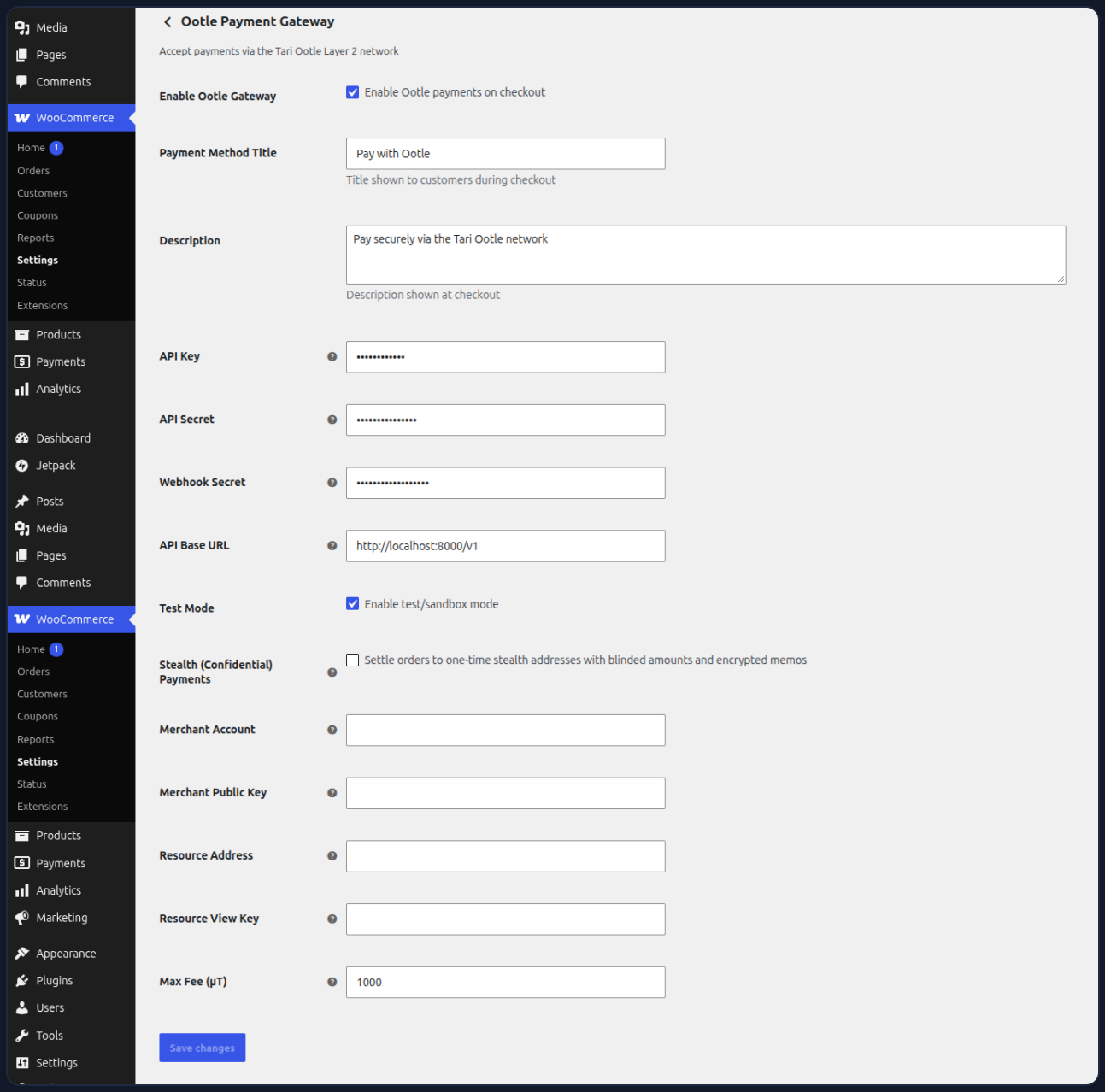


FIGURE 3 — THE OOTLE PAYMENT GATEWAY SETTINGS PAGE (ALL FIELDS, INCLUDING THE SIX STEALTH FIELDS)



FIELD	WHAT TO ENTER	NOTES
Enable Ootle Gateway	Check the box	REQUIRED off by default
Payment Method Title	Pay with Ootle	Shown to customers at checkout
Description	Pay securely via the Tari Ootle network	Shown under the title
API Key	The `tw_...` key from section 4	Paste exactly — no quotes, no extra spaces
API Secret	Leave blank unless signing is enabled	Only needed if you turn on HMAC request signing
Webhook Secret	A random 32+ character string you generate	See 6.2 below
API Base URL	`http://your-daemon-host:5100`	Your wallet daemon JSON-RPC endpoint
Test Mode	Check for testnet	Disables SSL verification for local/self-signed daemons

6.1 · FINDING YOUR API BASE URL

DAEMON LOCATION	API BASE URL VALUE
Same machine as WordPress	`http://127.0.0.1:5100`
Separate server on your LAN	`http://192.168.x.x:5100`
Remote VPS with a domain	`https://ootle.yourdomain.com` (reverse-proxy the daemon)

PRODUCTION MUST USE HTTPS

In production, always front the daemon with a reverse proxy (nginx or Caddy) and a valid certificate, so the API key is never sent over plain HTTP. Test Mode disables SSL verification — it must be **off** in production.

6.2 · GENERATE A WEBHOOK SECRET

Run this in your terminal and paste the output into the Webhook Secret field. This secret signs outgoing webhook payloads so the plugin can prove they came from a trusted source. It is never sent to the wallet daemon.

SHELL GENERATE THE WEBHOOK SECRET Copy

```
openssl rand -hex 32
```

7 Connect the webhook

Without a webhook, orders stay in "pending" forever — the plugin never learns the payment succeeded. Your indexer listener (or any monitoring service) must POST to the endpoint below whenever an on-chain payment event matches an order.

ITEM	VALUE
Webhook URL	`https://yoursite.com/?wc-api=ootle_webhook`
Method	`POST`
Content-Type	`application/json`
Auth header	`X-Ootle-Signature: HMAC-SHA256 of the raw body, keyed with your webhook secret`

The payload carries an event type and a session id. The plugin looks up the order by session id and updates its status. Six event types are supported:

EVENT	EFFECT ON THE ORDER
`payment.completed`	COMPLETED `payment_complete()` — transaction hash logged
`payment.failed`	FAILED order marked failed
`payment.cancelled`	CANCELLED order marked cancelled
`payment.refunded`	REFUND WooCommerce refund created, hash stored
`dispute.created`	NOTE dispute note added to order
`dispute.resolved`	RESOLVED order updated per ruling

> **NO LISTENER YET?**
>
> Orders remain pending, which is expected. The payment flow still works — you complete orders manually from WooCommerce → Orders until the listener is running.

---

8 Test a payment

8.1 · ADD A TEST PRODUCT

1 **Products → Add New**

Name it "Test Item" and set a regular price of 1.00.

2 **Publish**

The product is now purchasable.

8.2 · PLACE A TEST ORDER

1 **Add the test product to the cart**

Visit the shop page and add it.

2 **Go to checkout**

Confirm
"Pay with Ootle"
appears as a payment option — if it is missing, re-check that Enable Ootle Gateway is checked and saved.

3 **Select it and place the order**

The order should redirect to your Ootle payment URL.

4 **Check the order status**

Open WooCommerce → Orders and confirm the status is
pending
and the notes contain the payment URL.

8.3 · TROUBLESHOOT CONNECTION FAILURES

SYMPTOM	LIKELY CAUSE	FIX
cURL error 35: TLS connect error	Self-signed cert on a local daemon	`Enable Test Mode in plugin settings`
cURL error 7: connection refused	Daemon not running, or wrong host/port	`Start the daemon; verify API Base URL and port`
API error 401	API key wrong, mistyped, or revoked	`Re-copy the key; check it was not revoked`
API error 403: InsufficientPermissions	API key missing a required scope	`Add the scope and mint a new key`
Order stuck on pending	No webhook listener running	`Set up the listener (§7) or complete orders manually`

9 Going to production

9.1 · PRE-LAUNCH CHECKLIST

- ✓ **Switch the network** — restart the daemon with `--network mainnet`.
- ✓ **Turn off Test Mode** — uncheck it in plugin settings so SSL verification is enforced.
- ✓ **Update the API Base URL** — point at your production daemon endpoint.
- ✓ **Use HTTPS** — front the daemon with a reverse proxy and a valid certificate.
- ✓ **Rotate the API key** — mint a production key with production scopes; revoke the testnet key.
- ✓ **Set a key expiry** — long but finite, so a leaked key eventually dies.
- ✓ **Fund the account** — confirm the merchant account has mainnet TARI for fees.
- ✓ **Place one live order** — a small real purchase before announcing the gateway.

9.2 · SECURITY RULES THAT ARE NOT OPTIONAL

RULE	WHY
Never run an unauthenticated daemon with real funds	Anyone who can reach the port can drain the account
Never commit the API key or webhook secret	Version control history is permanent
Use the smallest scope set possible	A leaked read-only key cannot drain the account
Set key expiry	Limits how long a leaked key is useful
Revoke unused keys	Fewer live keys means less attack surface
Firewall the daemon port	Only the WordPress server should reach port 5100

10 Reference — settings at a glance

PLUGIN FIELD	TESTNET VALUE	PRODUCTION VALUE
Enable Ootle Gateway	YES	YES
Payment Method Title	Pay with Ootle	Pay with Ootle
Description	Pay securely via the Tari Ootle network	Pay securely via the Tari Ootle network
API Key	`tw_...` (testnet key)	`tw_...` (production key)
API Secret	blank unless signing enabled	blank unless signing enabled
Webhook Secret	`openssl rand -hex 32` output	different random output
API Base URL	`http://127.0.0.1:5100`	`https://ootle.yourdomain.com`
Test Mode	YES skips SSL verify	NO enforces SSL verify

11 Where to get help

RESOURCE	URL
Ootle documentation	ootle.tari.com
API key guide	ootle.tari.com/guides/agent-api-keys
Wallet setup guide	ootle.tari.com/guides/setup-a-wallet
Transaction overview	ootle.tari.com/guides/transaction-overview
Source code & issues	github.com/tari-project/tari-ootle
Releases (binaries)	github.com/tari-project/tari-ootle/releases
Community forum	community.tari.com

END OF GUIDE

If a step fails, start from the symptom table in section 8.3 before escalating. Most first-time failures are a wrong port, an unchecked Test Mode box, or a missing API key scope — all three are fixable in under a minute.